

Uso do Solo v3 — Enriquecimento dos Requisitos + Cascata Completa (pela interface do LangNet)

Projeto: Gestão Municipal de Uso do Solo — instância c4871aaf (v3) **Data:** 28/08/2026 **Regra do teste:** todas as correções feitas **pela interface do LangNet** (Assistente de Requisitos → Especificação → Modelo de Dados → Agent-Task Spec → tasks.yaml → Código). Zero edição manual de artefato. Corrigir o gerador/prompts (o produto) é legítimo.

1. Contexto: o que aconteceu entre as versões (sem código de commit)

Você percebeu certo: **existia uma versão que tinha o calculador urbanístico e o v3 não tinha**. Não é bug de mistura — foi **um galho da árvore que não voltou pro tronco**. Em linguagem de versões:

- **Versão “Operacional” (a que virou o v3, projeto c4871aaf)**. Nasceu focada na **operação municipal**: cadastros, licenciamento/alvará, protocolos, fiscalização, notificações, dashboard de gestão, perfis de usuário. É rica em **processo administrativo**, mas o **cálculo urbanístico** (CA, TO, recuos, gabarito, APP, reserva legal, declividade) aparece só como menção — **não há requisito que mande o sistema calcular**.
- **Versão “Calculador” (um projeto irmão, gerado depois que você carregou as leis de uso do solo)**. Quando você mandou **acrescentar a legislação de uso do solo**, esse enriquecimento foi aplicado **sobre um galho paralelo**, não sobre o v3. Essa versão ganhou os **eixos de cálculo** de verdade (FR-001 parâmetros por zona, FR-003 CA e TO, FR-004 recuos e gabarito, FR-006 APP, FR-007 reserva legal, FR-008 declividade, importação de Shapefile), mas **não herdou** todo o operacional da outra.
- **Onde foi feita a “merda”**. No momento de cascadear o **v3**, ele foi montado a partir da versão **Operacional** (a que não tinha cálculo) e **puxou a especificação do galho errado** — por isso o app do v3 saiu sem calculadora. O enriquecimento com as leis **existiu**, mas **num galho que o v3 nunca incorporou**. Ninguém apagou requisito; o v3 **nunca recebeu** os requisitos de cálculo.
- **Conclusão / decisão**. As duas versões **divergiram** (nenhuma é superconjunto da outra): uma tem o operacional completo, a outra tem o cálculo completo. A correção certa é **unir os dois galhos** — pegar **tudo o que o v3 já tem de operacional** e **somar por cima os eixos de cálculo** da versão calculadora — e então **re-cascadear** o v3 inteiro (Spec → Modelo de Dados → ATS → tasks.yaml → Código). É exatamente o que este relatório documenta, **feito pela interface**.

2. Etapa 1 — Enriquecer os Requisitos do v3 (Assistente de Requisitos)

Como foi feito pela UI: abri o **Assistente de Requisitos** do projeto v3 (que estava vazio), subi os **dois documentos-fonte** e disparei a **Análise** com uma instrução de **unificação** — o próprio pipeline de requisitos do LangNet funde os dois (o endpoint `analyze-batch` concatena todos os documentos e manda para o workflow de análise).

Documentos subidos: - `requisitos_v3_operacional.md` — a base **operacional** do v3 (cadastros, licenciamento, fiscalização, dashboard...). - `requisitos_v2_calculador.md` — a versão **com os eixos de cálculo** (CA/TO, recuos, gabarito, APP, reserva legal, declividade, Shapefile).

Instrução de unificação dada ao agente (resumo): preservar integralmente o operacional do 1º documento; adicionar por cima os eixos de cálculo do 2º; não remover nada para caber o cálculo; renumerar FR-XXX de forma contínua e sem duplicatas.

Figura 2.1 — Os dois documentos carregados (Pendente), a instrução de unificação e a Pesquisa Web desligada, prontos para **Iniciar Análise**. `shots/05-instrucoes-config.png`

Figura 2.2 — Análise iniciada (o Assistente processa e funde os dois documentos). `shots/06-analise-iniciada.png`

Resultado da fusão — ACHADO IMPORTANTE (limitação do produto): A análise concluiu e salvou a **versão 1** (29 KB), mas o pipeline de "Iniciar Análise" **resumiu em vez de fundir**. Ele produziu **14 FRs genéricos** e: - **colapsou todos os eixos de cálculo** num único requisito vago **FR-008 "Eixos de Cálculo Urbanístico"** ("o sistema deve incluir módulos de cálculo urbanístico") — CA, TO, recuos, gabarito, reserva legal, declividade, Shapefile sumiram como requisitos concretos; - **transformou a minha instrução** de unificação num requisito bobo **FR-007 "Integração de Documentos"**; - **comprimiu o operacional** para 6 FRs genéricos.

Conclusão: o "Iniciar Análise" (pipeline de 12 passos) é um **sumarizador**, não um **fusor** — alimentar dois documentos de requisitos já estruturados faz ele re-sintetizar e perder detalhe dos dois lados. **O LangNet não tem hoje um "fundir duas versões de requisitos preservando detalhe".**

Ferramenta certa = o chat "Refinar". Diferente da análise, o endpoint de refino é **uma única chamada dirigida** cujo prompt manda "COPIE todo o conteúdo original + aplique só as mudanças; PRESERVE os IDs; só os requisitos diretamente afetados mudam" — e ele **relê os dois documentos originais**. Ou seja, dá pra pedir "expanda o FR-008 em FRs concretos por eixo (CA/TO/recuos/gabarito/APP/reserva legal/declividade/Shapefile), preservando o operacional" **sem** resumir o resto. Esse é o caminho de correção (pela UI).

Correções de infra feitas no backend (produto, legítimas): - o backend travou inteiro (até /health pendurava) por causa do **pool de conexões**: ele abria **20 conexões de uma vez** contra o banco DDNS remoto e estourava timeout no init do pool — e o fallback de conexão direta **não cobria falha de init** (só exaustão). Reduzi o pool para **5** e fiz o fallback direto cobrir **também** a falha de init do pool.

BLOQUEIO ATUAL (28/08, ~13h20): o host remoto `camerascasas.no-ip.info` **caiu** — ping 100% de perda; **:3308 (MariaDB) e :1234 (LM Studio) inacessíveis**. Como esse host provê **o banco e o LLM**, o refino e toda a cascata ficam **parados até o host voltar**. Nada a fazer do meu lado além de retomar assim que ele responder.

Etapa 1b — Refino pela UI (RESULTADO: SUCESSO)

Assim que o host voltou, refinei pela UI (chat "Refinar"). Como o refino **relê os dois documentos originais** e só altera o que é pedido, ele **expandiu o FR-008 genérico em requisitos concretos por eixo** e **preservou** o resto. **Versão final = 4** (57 KB, **37 FRs**), com o próprio changelog do documento registrando "Unificação dos documentos v3_operacional e v2_calculador".

Eixos de cálculo agora como **FRs individuais com fórmula/critério**:

FR	Requisito
FR-014	Parâmetros urbanísticos por zona (ca_maximo, to_maxima, recuos, gabarito, área mín.)
FR-015	Zoneamento poligonal (carregar/visualizar/editar)
FR-016	Cálculo de CA (Coeficiente de Aproveitamento) + validação contra a zona
FR-017	Cálculo de TO (Taxa de Ocupação) + validação
FR-018	Cálculo e validação de recuos (frontal/lateral/fundos)
FR-019	Validação de gabarito (altura máxima)
FR-020	Delimitação e sobreposição de APP
FR-021	Cálculo de Reserva Legal
FR-022	Avaliação de declividade do terreno
FR-023	Consulta de conformidade urbanística consolidada (retorna todos os cálculos)
FR-024	Importação de geodados (Shapefile/GeoJSON/KML)

Operacional + geoespacial preservados (FR-001..013, FR-025..030): RBAC, versionamento de legislação, notificações, mapa interativo, geoprocessamento, IDE Sisema.

Figura 2.3 — Documento de requisitos unificado (v4), visualizado na UI. `shots/12-doc-refinado.png`

Etapa 1 CONCLUÍDA. Segue a cascata a partir da versão de requisitos 4.

Plano da cascata (em execução)

- Especificação** ← requisitos v4 (UCs de cálculo + wireframes ricos).
- Modelo de Dados** PostGIS (colunas `ca_maximo`, `area_terreno`, `area_construida`, `ca_calculado...`).
- UI Spec** → **ATS** (cobertura total UC + fidelidade de cálculo) → **tasks.yaml** (traceability) → **Código** (renderizadores ricos + calculadora).
- Deploy** + **E2E** provando CA/TO; **PDF final**.

3. Etapa 2 — Especificação (re-gerada a partir dos requisitos unificados)

Disparei a geração pela UI (página **Especificação** → selecionei os **Requisitos v4** → **Gerar Especificação**), com instrução para cobrir os 4 eixos (urbanístico/ambiental/**cálculo-conformidade**/operacional) e o modelo de dados com colunas de cálculo.

Resultado: documento com **14 seções, 42.471 chars**, status **completed**, com **proveniência gravada** (requirements_session = dc66b1e7, v4). **10 casos de uso**, cobrindo os eixos de cálculo:

- **UC-001: Consulta de Conformidade Consolidada** — o UC central: dado o lote/edificação, calcula CA, TO, recuos, gabarito, APP, etc. e devolve conforme/não-conforme.
- UC-003 Delimitação e verificação de APP · UC-005 Edição de parâmetros urbanísticos por zona · UC-009 Dashboard de Conformidade Municipal.

Verificação automática do conteúdo: presentes **CA, TO, recuos, gabarito, APP, reserva legal, declividade** e as **colunas de cálculo** ca_maximo, area_terreno, area_construida, ca_calculado, além de geometry. A própria spec anota que o cálculo consolidado (UC-001) exige índices espaciais + cache de parâmetros de zona.

Figuras 3.x — seleção da v4, preview e disparo da geração. shots/13..17-spec-*.png

4. Etapa 3 — Modelo de Dados (PostGIS, com colunas de cálculo)

Gerei o Modelo de Dados pela UI (página **Modelo de Dados** → spec de origem = a do v3 → **DBMS PostgreSQL** → **Gerar**). Resultado inicial: **PostGIS OK** (extensão, geometry(...,4674), índices GiST), **10 tabelas** operacionais/geoespaciais — **mas o gerador dropou as colunas de cálculo** (a spec tinha ca_maximo, area_terreno, ca_calculado; o DM não trouxe). Mais um ponto onde o cálculo se perdia rio abaixo.

Correção pela UI (chat "Refinar" do DM): pedi explicitamente as entidades/colunas de cálculo. O refino re-emitiu os artefatos preservando o operacional. **Versão 2** agora tem:

- **Novas tabelas de cálculo:** parametros_urbanisticos (ca_maximo, to_maxima, recuos, gabarito, área mínima por zona), imoveis (area_terreno), edificacoes (area_construida, pavimentos, altura), apps, reservas_legais, **calculos_conformidade** (ca_calculado, to_calculado, conforme, resultado).
- **Mantém** PostGIS (geometry SRID 4674) e as tabelas operacionais (municipios, zoneamentos, legislacoes, usuarios, empreendimentos, consultas).

Figuras 4.x — geração PostGIS, refino do cálculo e schema final. shots/18..23-dm-*.png

Observação de produto: o cálculo se perde em CADA passagem de LLM (requisitos→spec ok, mas spec→DM dropou). O padrão de correção que funciona é o **chat "Refinar" com instrução concreta** (colunas/fórmulas explícitas) — igual deu certo nos requisitos.

4b. UI Spec (telas ricas, não CRUD)

Gerada pela UI a partir da Especificação do v3. **10 telas**, com o catálogo de componentes rico ligado ao cálculo:

Tela	Componentes ricos
Resultado de Conformidade	mapa + 5 metric-cards (CA, TO, recuos calculados) + gráfico + timeline
Cálculos Urbanísticos	mapa + metric-cards
Análise Ambiental: APP	mapa + metric-cards + gráfico + timeline
Importar Geodados	file-upload + file-preview + mapa (Shapefile)
Dashboard de Conformidade Municipal	4 KPIs + gráfico + mapa

Tipos usados: map, chart, metric-card, file-upload, file-preview, kanban, timeline — arquétipos ricos, não o molde de formulário. Mockups PNG renderizados (4,9 MB). shots/25..26-uispec-*.png.

5. Etapa 4 — Agent-Task Spec (cobertura total dos UCs + fidelidade de cálculo)

Gerado pela UI a partir da Especificação do v3 (botão **Gerar Agentes & Tarefas**). A 1ª tentativa **falhou** (o host oscilou no meio da chamada ao LLM); a 2ª concluiu.

Resultado: 29 KB, **9 agentes, 12 tarefas, cobrindo os 10 UCs** (UC-001...UC-010, sem lacuna — o laço de cobertura do gerador garante ≥ 1 task por UC). **Fidelidade de cálculo** confirmada: a ATS traz as fórmulas ($CA = \text{area_construida} / \text{area_terreno}$, comparação com ca_maximo , recuos, gabarito, conformidade) — não é task genérica de CRUD.

Figuras 5.x — shots/27..29-ats-*.png.

6. Etapa 5 — tasks.yaml (com `traceability: {uc, fr}`)

Gerado pela UI (rota `/yaml-generation`, base = ATS do v3). **12 tasks, status completed** (20 KB), com o campo `execution` roteando e `traceability` por task. O destaque é a **task de cálculo real** — o calculador deixou de ser abstrato e virou passos executáveis:

```
calculate_ca_to:
  traceability: { uc: UC-002, fr: [FR-016, FR-017] }
  execution: deterministic
  agent: urban_calc_agent
  steps:
    1) SELECT i.area_terreno, e.area_construida, e.area_projecao
       FROM imoveis i JOIN edificaciones e ON e.imovel_id = i.id WHERE i.id = %s
    2) ca_calc = area_construida / area_terreno ; to_calc = area_projecao / area_terreno
    3) SELECT p.ca_maximo, p.to_maxima FROM zoneamentos z
       JOIN parametros_urbanisticos p ON z.id = p.zona_id
       WHERE ST_Contains(z.geometria, (SELECT geometria FROM imoveis WHERE id = %s))
    4-5) comparar ca_calc x ca_maximo e to_calc x to_maxima -> status = conforme|nao_conforme
  expected_output: { ca_calculado, to_calculado, status_ca, status_to }
```

- **Fórmula real** ($CA = \text{área construída} \div \text{área do terreno}$; TO idem), **lookup da zona por PostGIS** (`ST_Contains`), comparação de conformidade.
- **12 blocos traceability** (UC/FR por task); `execution: deterministic` nas de cálculo, `agent` nas de composição.
- Outras tasks: `import_validate_geodata` (Shapefile), `generate_compliance_report`, `query_point_zone_app`, `generate_compliance_dashboard`, `simulate_scenario`.

Figuras 6.x — shots/31..33-tasksyaml-*.png.

7. Etapa 6 — Código (renderizadores ricos + calculadora CA/TO)

Gerado pela UI (**Gerar Código**, base = `agents.yaml` + meu `tasks.yaml` 166d5f85). **81 arquivos, status completed**. O calculador atravessou até aqui: `- ws-server/tasks.yaml` — a task `calculate_ca_to` com a **fórmula real** ($ca_calc = \text{area_construida} / \text{area_terreno}$), lookup de zona por `ST_Contains` e comparação de conformidade. `- frontend/src/screens/CalculosConformidadeCrud.jsx` — tela do cálculo. `- db/schema.sql` — PostGIS com as tabelas de cálculo. `- ws-server/adapters.py` (83 KB) — **60 funções CRUD *_deterministic** (todas as 12 tabelas).

ACHADO (limitação real do gerador — última milha): as tasks de **COMPUTAÇÃO multi-passo** (`calculate_ca_to`, `validate_setbacks_height`, `calculate_legal_reserve`, `simulate_scenario`) receberam apenas **wrappers input_func/output_func** — **não** um `calculate_ca_to_deterministic` com o SQL+divisão. O roteador do `ws-server` é `deterministic-first`: sem `<task>_deterministic`, a task **cai no agente**. Ou seja, **o gerador traduz CRUD para Python determinístico, mas ainda não emite a COMPUTAÇÃO complexa** (o pendente conhecido: "emissão estruturada p/ computação complexa"). No runtime, `calculate_ca_to` executaria pelo agente (LLM), não pelo caminho exato.

Figuras 7.x — shots/34..35-codegen-*.png.

Decisão: deploy + E2E para medir o comportamento real (agente calcula vs. erra) e, se preciso, corrigir o gerador para emitir a computação determinística.

8. Deploy + E2E — o que roda e o que falta (honesto)

Exportei os 81 arquivos e preparei o deploy PostGIS. Ao inspecionar o app gerado antes de rodar, encontrei **dois bloqueios de runtime** para a task de cálculo `calculate_ca_to`:

1. **Sem função determinística de COMPUTAÇÃO.** `adapters.py` tem 60 CRUDs `*_deterministic`, mas **não** um `calculate_ca_to_deterministic` com o SQL+divisão. O roteador do `ws-server` é `deterministic-first`: sem essa função, a task cai no agente. → **limitação do gerador** (não emite computação multi-passo; só CRUD).
2. **agents.yaml incoerente.** O code-gen foi feito com o `agents.yaml` **antigo** (session f9bb86cb, pré-calculadora), que define `consulta_agent`, `legislacao_importer_agent`, ... — mas o `tasks.yaml` novo referencia `urban_calc_agent`, `environmental_calc_agent`, `compliance_engine_agent`. **Os agentes de cálculo não existem** no `agents.yaml`. → o fallback para agente **também** falharia. (Causa: não há um caminho limpo na UI para regenerar SÓ o `agents.yaml` a partir do novo ATS sem também regenerar o `tasks.yaml`.)

Conclusão honesta: o app, como gerado agora, **não executa** o `calculate_ca_to` (sem função determinística + agente indefinido). **NÃO** afirmo que a calculadora roda E2E.

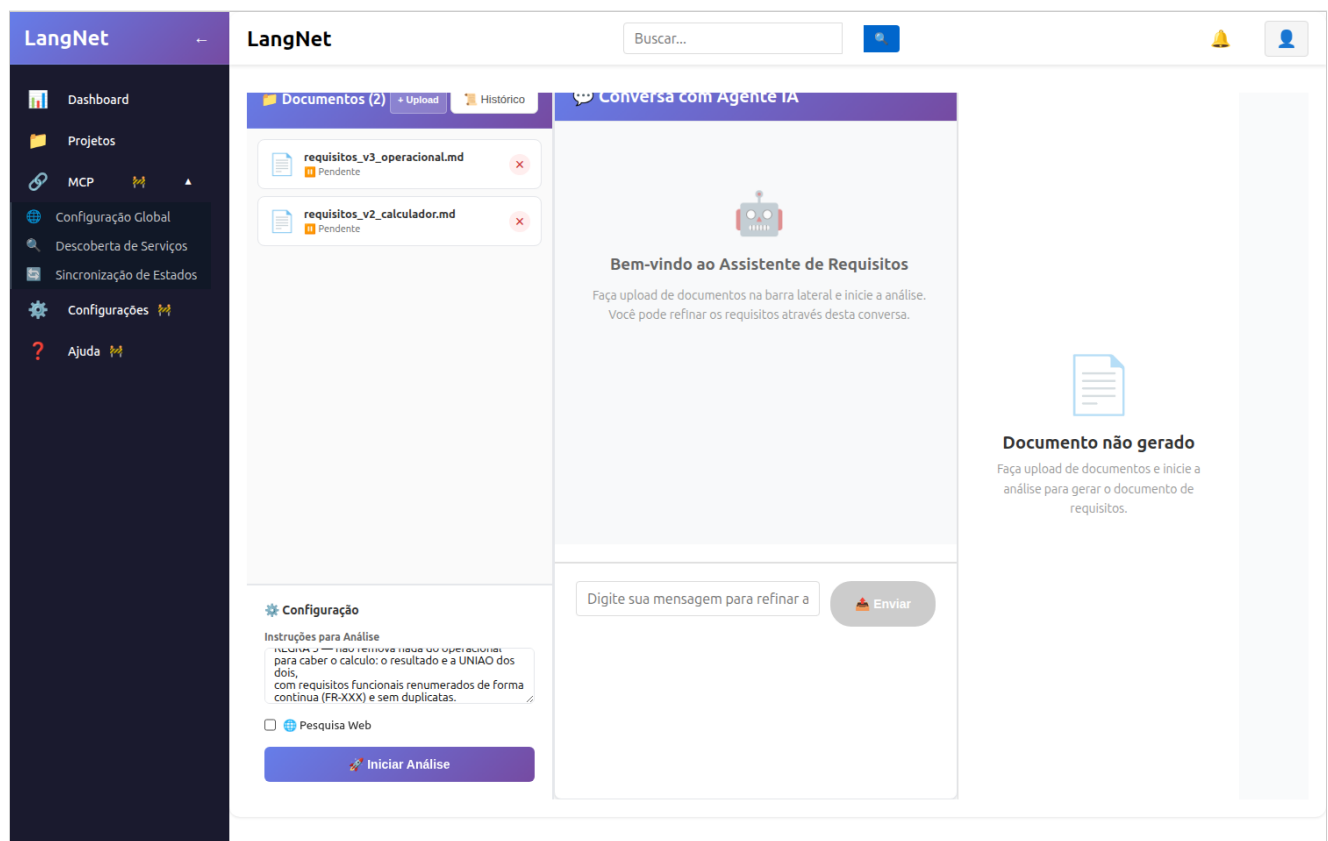
O que ESTÁ provado (o mérito desta cascata)

O calculador — que **não existia** no v3 — agora atravessa **todo** o pipeline, verificado artefato a artefato no banco: Requisitos v4 (FR-016 CA, FR-017 TO, ...) → Especificação (UC-001 Conformidade) → Modelo de Dados PostGIS (`parametros_urbanisticos`, `calculos_conformidade`, `ca_calculado`) → UI Spec (tela de Resultado de Conformidade com `metric-cards`) → ATS (task com fórmula) → **tasks.yaml** (`calculate_ca_to` com `ca=area_construida/area_terreno`, `ST_Contains`, **comparação**) → Código (81 arquivos, `ws-server/tasks.yaml` com a fórmula, tela `CalculosConformidadeCrud.jsx`, docs/`RASTREABILIDADE.md`).

Para a calculadora RODAR (2 correções concretas de produto)

1. **Gerador — emitir computação determinística:** estender o parser (`_parse_task_description_to_python/_emit_sql_step`) para traduzir tasks de COMPUTAÇÃO multi-passo (capturar SELECT em variáveis → aritmética `ca=area_construida/area_terreno` → 2º SELECT com join espacial → comparação → JSON) em `calculate_ca_to_deterministic`. Hoje só CRUD é emitido.
2. **Regenerar `agents.yaml` a partir do novo ATS** (e um caminho de UI para regenerar agents sem sobrescrever o `tasks.yaml`), para os agentes referenciados existirem.

Anexo — Galeria de evidências (telas)



Etapa 1 — os dois documentos-fonte + instrução de unificação (Pesquisa Web off)

Comparação de Alterações					
Antes do Refinamento			Depois do Refinamento		
1	# Documento de Requisitos		1	# Documento de Requisitos	
2	## Análise de Requisitos - Projeto c4871aaf-3c8c-41d3-8ca7-6c3e22189731		2	## Análise de Requisitos - Projeto c4871aaf-3c8c-41d3-8ca7-6c3e22189731	
3			3		
4	---		4	---	
5			5		
6	- **Versão:** 2.1		6	+ **Versão:** 2.2	
7	- **Data:** 2026-08-28 13:25:00		7	+ **Data:** 2026-08-28 13:45:00	
8	**Status:** Draft		8	**Status:** Draft	
9			9		
10	---		10	---	
11			11		
12	## 1. Informações do Projeto		12	## 1. Informações do Projeto	
13			13		
14	### 1.1 Visão Geral		14	### 1.1 Visão Geral	
15	**Nome do Projeto:** Plataforma Integrada de Gestão Municipal de Uso do Solo e Cálculos Urbanísticos		15	**Nome do Projeto:** Plataforma Integrada de Gestão Municipal de Uso do Solo e Cálculos Urbanísticos	
16			16		
17	**Descrição:**		17	**Descrição:**	
18	Plataforma unificada que integra a gestão operacional municipal (cadastros, licenciamento, alvarás, fiscalização, protocolos, notificações, relatórios) com os eixos de cálculo urbanístico e ambiental (CA, TO, recuos, gabarito, APP, Reserva Legal, declividade, zoneamento poligonal, importação de geodados e geração de laudos). O sistema trata ambos os conjuntos de funcionalidades como uma única aplicação evolutiva, onde a base operacional sustenta os cálculos técnicos e os resultados de conformidade alimentam os processos administrativos.		18	Plataforma unificada que integra a gestão operacional municipal (cadastros, licenciamento, alvarás, fiscalização, protocolos, notificações, relatórios) com os eixos de cálculo urbanístico e ambiental (CA, TO, recuos, gabarito, APP, Reserva Legal, declividade, zoneamento poligonal, importação de geodados e geração de laudos). O sistema trata ambos os conjuntos de funcionalidades como uma única aplicação evolutiva, onde a base operacional sustenta os cálculos técnicos e os resultados de conformidade alimentam os processos administrativos.	
19			19		
20	**Objetivo:**		20	**Objetivo:**	
21	Unificar a gestão operacional municipal com os eixos de cálculo		21	Unificar a gestão operacional municipal com os eixos de cálculo	

Etapa 1b — documento unificado (v4): descrição integrada operacional + eixos de cálculo

Document Analysis - 2 files - v4

Sessão: Document Analysis - 2 files | Versão: v4

Preview do Documento

Documento de Requisitos

Análise de Requisitos - Projeto c4871aaf-3c8c-41d3-8ca7-6c3e22189731

Versão: 2.2 Data: 2026-08-28 13:45:00 Status: Draft

1. Informações do Projeto

1.1 Visão Geral

Nome do Projeto: Plataforma Integrada de Gestão Municipal de Uso do Solo e Cálculos Urbanísticos

Descrição: Plataforma unificada que integra a gestão operacional municipal (cadastros, licenciamento, alvarás, fiscalização, protocolos, notificações, relatórios) com os eixos de cálculo urbanístico e ambiental (CA, TO, recuos, gabarito, APP, Reserva Legal, declividade, zoneamento poligonal, importação de geodados e geração de laudos). O sistema trata ambos os conjuntos de funcionalidades como uma única aplicação evolutiva, onde a base operacional sustenta os cálculos técnicos e os resultados de conformidade alimentam os processos administrativos.

Objetivo: Unificar a gestão operacional municipal com os eixos de cálculo urbanístico e ambiental em uma única plataforma coesa, garantindo que todo requisito operacional do documento base (requisitos_v3_operacional) seja preservado integralmente, e que os módulos de cálculo e análise espacial do documento complementar (requisitos_v2_calculador) sejam adicionados sem duplicatas, com renomeação contínua e coerente dos IDs.

Seleção: Document Analysis - 2 files v4

cancela

✓

Selecionar este Documento

Etapa 2 — seleção da versão 4 de requisitos para gerar a Especificação

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Docs

Complementares (2)

+ Upload

Histórico

Requisitos

Base: Document Analysis - 2 files (v4)

20260828_125021_requisitos_v2_C...

Complementar

20260828_125019_requisitos_v3_...

Complementar

Configuração

Instruções Adicionais

Wireframe dos Casos de Uso

Gerar Especificação

Revisar Especificação

GERANDO ESPECIFICAÇÃO INICIAL...

Aguarde, isso pode levar 1-3 minutos

Processando... 0% 0/0 tarefas

Conversa com Agente IA

Sistema 13:55

2 documento(s) complementar(es) incluído(s)

Agente IA 13:55

Sessão iniciada! Gerando especificação funcional. Isso pode levar alguns minutos...

Sistema 13:55

GERANDO ESPECIFICAÇÃO INICIAL...

Aguarde, isso pode levar 1-3 minutos

Digite sua mensagem...

Analisar

Refinar

Geração de especificação iniciada!

Especificação não gerada

1. Selecione um documento de requisitos (Requisitos)

2. Opcionalmente adicione docs complementares

3. Clique em "Gerar Especificação"

Etapa 2 — geração da Especificação iniciada

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Modelo de Dados

Gera schema SQL, models.py (SQLAlchemy+Pydantic) e migrations Alembic a partir da especificação. Extrai entidades e relacionamentos automaticamente.

Especificação

Historico

Spec v4 (573139a3...)

Origem: Especificação 573139a3...

Configuração

Banco de dados (DBMS)

PostgreSQL

Instruções Adicionais

Ex: Focar em cenários de erro, detalhar validações...

Regenerar do zero

Revisar

Ocultar chat

Refinar via chat

Modelo atualizado.

Ex: adicione tabela comentarios

Enviar

Rascunho

Versão v1

DBMS POSTGRESQL

12 tabelas

Aprovar

Validação (score 85/100) — 6 problemas

[high]

empreendimentos: Tabela isolada sem relação (FK) com 'municipios' ou 'imoveis', dificultando a rastreabilidade dos dados.

[high]

consultas: Tabela isolada sem relação (FK) com 'imoveis' ou 'empreendimentos', tornando impossível vincular o resultado da consulta ao objeto analisado.

[medium]

calculos_conformidade: Coluna 'imovel_id' possui índice, mas a tabela permite múltiplos registros por imóvel (histórico). Se a intenção for apenas o último cálculo, falta um constraint ou lógica de exclusividade; se for histórico, o índice é correto, mas a ausência de FK para 'zoneamentos' impede saber qual zona foi usada no cálculo.

[medium]

legislacoes: Coluna 'nome' possui índice único, o que pode ser restritivo se houver leis com nomes idênticos em contextos diferentes, ou desnecessário se o nome não for chave de negócio.

[low]

apps: Coluna 'tipo' não possui índice, o que pode impactar performance em filtros por tipo de APP.

Entidades

Schema SQL

models.py

Alembic

YAML

municipios

Entidade raiz com dados do municipio

id

nome

created_at

updated_at

UUID

VARCHAR(200)

TIMESTAMP

TIMESTAMP

NN DEF

NN

NN DEF

NN DEF

zoneamentos

Polígonos de zoneamento com parâmetros

id

municipio_id

nome

geometria

created_at

updated_at

UUID

UUID

VARCHAR(200)

GEOMETRY

TIMESTAMP

TIMESTAMP

NN DEF

NN

NN

NN

NN DEF

NN DEF

parametros_urbanisticos

Etapa 3 — Modelo de Dados após refino (calculador + PostGIS)

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Modelo de Dados

Gera schema SQL, models.py (SQLAlchemy+Pydantic) e migrations Alembic a partir da especificação. Extrai entidades e relacionamentos automaticamente.

Especificação

Historico

Spec v4 (573139a3...)

Origem: Especificação 573139a3...

Configuração

Banco de dados (DBMS)

PostgreSQL

Instruções Adicionais

Ex: Focar em cenários de erro, detalhar validações...

Regenerar do zero

Revisar

Ocultar chat

Refinar via chat

Modelo atualizado.

Ex: adicione tabela comentarios

Enviar

Rascunho

Versão v1

DBMS POSTGRESQL

12 tabelas

Aprovar

Validação (score 85/100) — 6 problemas

[high]

empreendimentos: Tabela isolada sem relação (FK) com 'municipios' ou 'imoveis', dificultando a rastreabilidade dos dados.

[high]

consultas: Tabela isolada sem relação (FK) com 'imoveis' ou 'empreendimentos', tornando impossível vincular o resultado da consulta ao objeto analisado.

[medium]

calculos_conformidade: Coluna 'imovel_id' possui índice, mas a tabela permite múltiplos registros por imóvel (histórico). Se a intenção for apenas o último cálculo, falta um constraint ou lógica de exclusividade; se for histórico, o índice é correto, mas a ausência de FK para 'zoneamentos' impede saber qual zona foi usada no cálculo.

[medium]

legislacoes: Coluna 'nome' possui índice único, o que pode ser restritivo se houver leis com nomes idênticos em contextos diferentes, ou desnecessário se o nome não for chave de negócio.

[low]

apps: Coluna 'tipo' não possui índice, o que pode impactar performance em filtros por tipo de APP.

Entidades

Schema SQL

models.py

Alembic

YAML

CREATE EXTENSION IF NOT EXISTS postgres;

CREATE TABLE "municipios" (

"id" uuid PRIMARY KEY DEFAULT gen_random_uuid(),

"nome" VARCHAR(200) NOT NULL,

"created_at" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,

"updated_at" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE "zoneamentos" (

"id" uuid PRIMARY KEY DEFAULT gen_random_uuid(),

"municipio_id" uuid NOT NULL,

"nome" VARCHAR(200) NOT NULL,

"geometria" geometry(geometry,4674) NOT NULL,

"created_at" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,

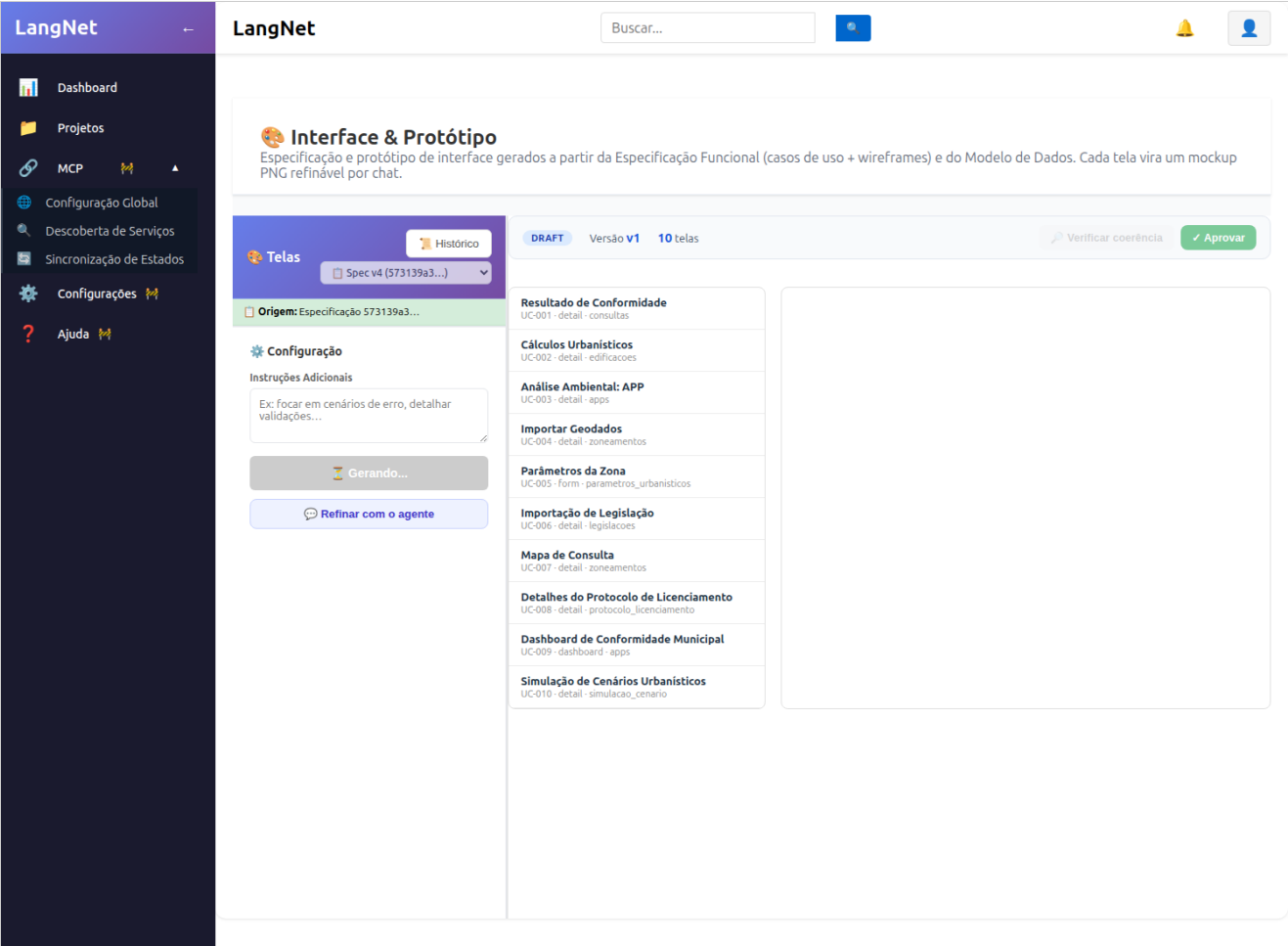
"updated_at" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY ("municipio_id") REFERENCES municipios(id) ON DELETE CASCADE

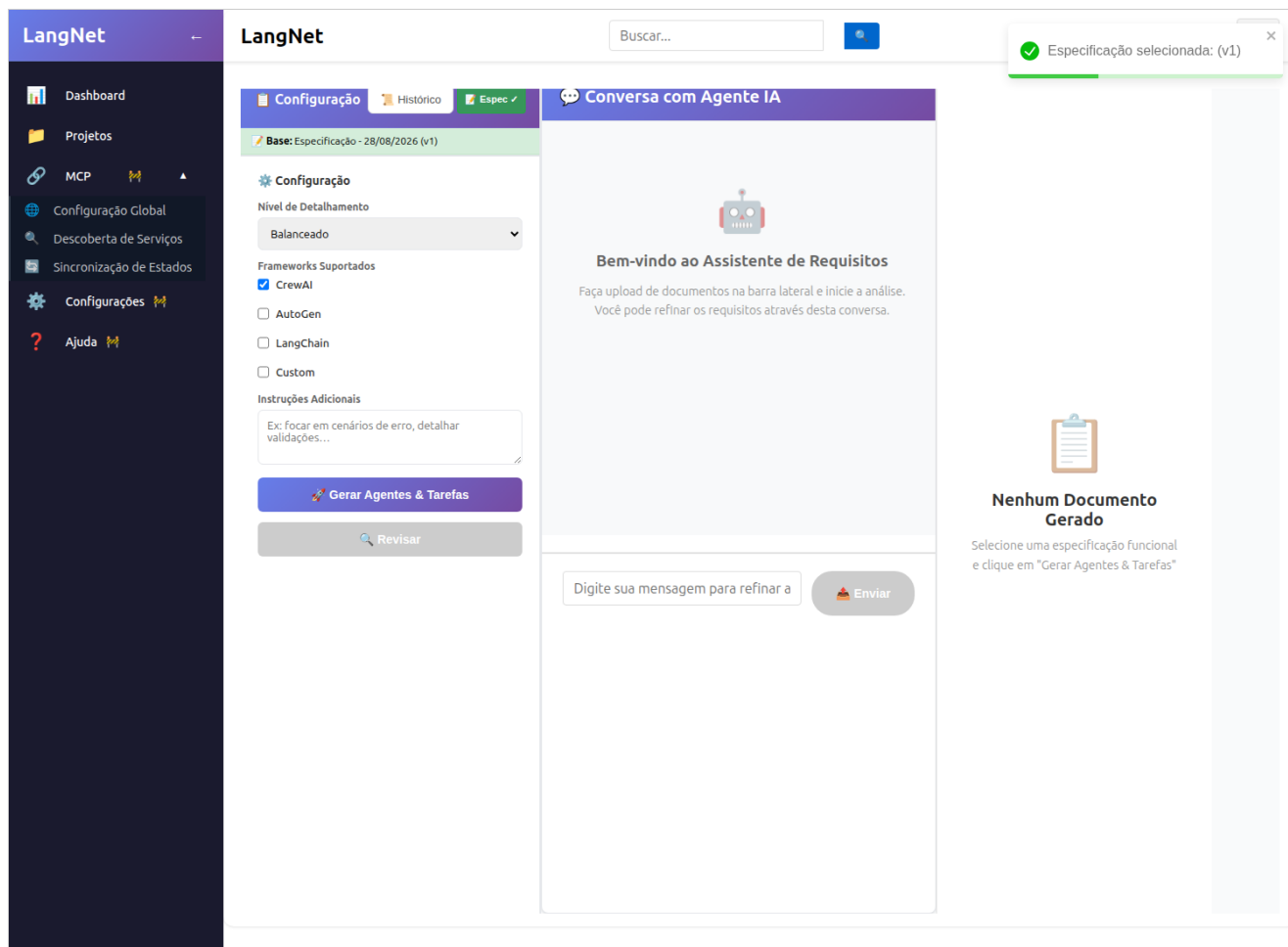
);

CREATE TABLE "parametros_urbanisticos" (

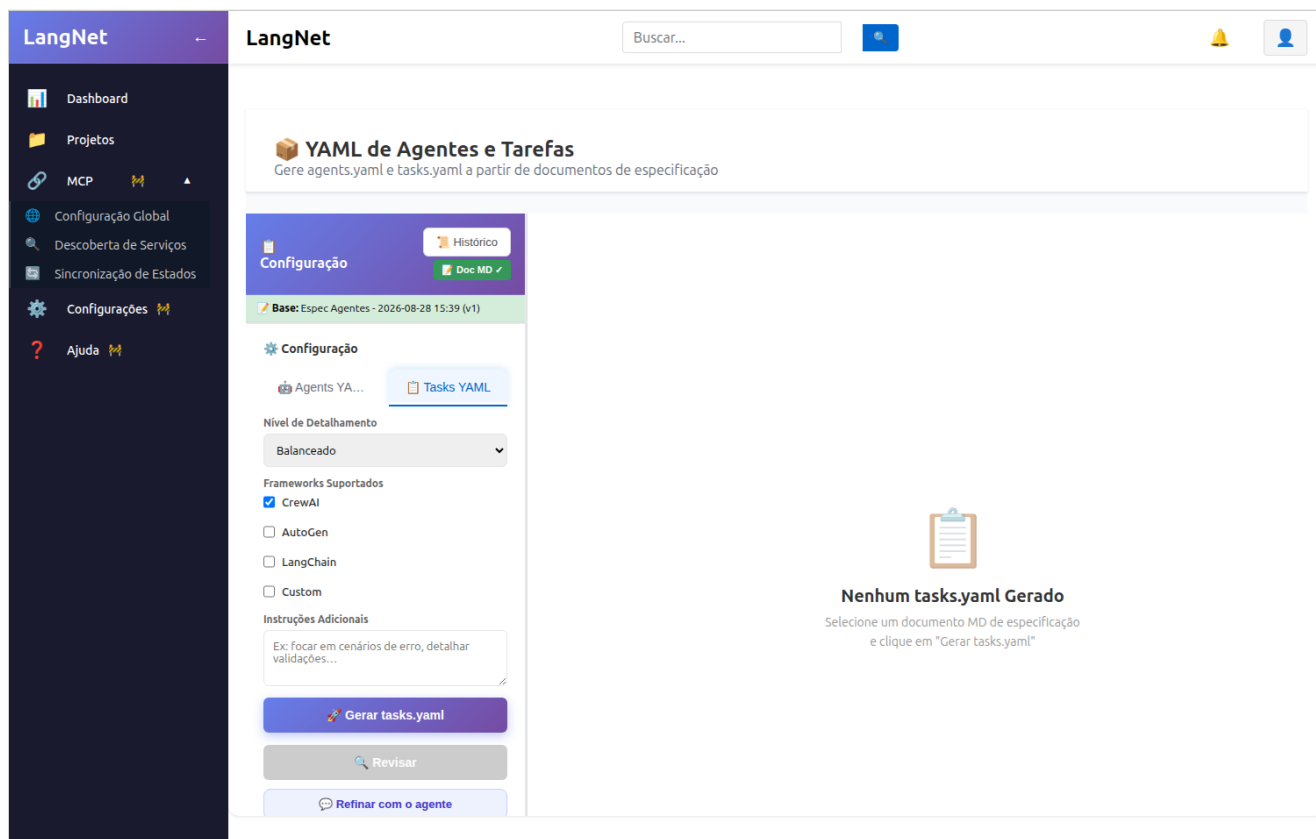
Etapa 3 — schema SQL PostGIS com as tabelas de cálculo



Etapa 3.5 — UI Spec: telas ricas (mapa/metric-cards/gráfico)



Etapa 4 — ATS: especificação selecionada para gerar agentes & tarefas



Etapa 5 — tasks.yaml gerado (12 tasks, com calculate_ca_to)

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Geração de Código

Geração do sistema agêntico Python (agents.yaml, tasks.yaml, tools.py, adapters.py, petri_net.json...) a partir das especificações. Refine com o agente, corrija warnings e execute o servidor.

Geração

Histórico

Nova geração

Sessão: code_gen_20260828_185616 (v1 - 81 arquivos - completed)

Configuração

SESSÕES (9)

code_gen_20260828_185616
v1 - 81 arquivos - completed

code_gen_20260827_161051
v1 - 87 arquivos - completed

code_gen_20260827_141244
v1 - 85 arquivos - completed

code_gen_20260827_140406
v1 - 85 arquivos - completed

Instruções Adicionais

Ex: focar em cenários de erro, detalhar validações...

Gerar Código

Refinar com o agente

ARQUIVOS (81)

ws-server/main.py

ws-server/websocket_server.py

ws-server/database_tool.py

ws-server/geoprocessamento_tool.py

ws-server/agis_bridge.py

ws-server/tools.py

ws-server/tools_std.py

ws-server/tools_ext.py

ws-server/adapters.py

ws-server/agents.yaml

ws-server/tasks.yaml

ws-server/petri_net.json

ws-server/requirements.txt

ws-server/env.example

ws-server/Dockerfile

README.md

backend/Dockerfile

backend/main.py

backend/requirements.txt

ws-server/main.py

```
1  """
2  Uso do Solo v3 (regeneracao gerador corrigido) - entrypoint
3  Sobe o servidor WebSocket na porta 5025 que recebe execute_task
4  e dispara a task CrewAI correspondente.
5  """
6  import asyncio
7  import os
8  from dotenv import load_dotenv
9
10 from websocket_server import run_websocket_server
11
12 load_dotenv()
13
14 PROJECT_NAME = "Uso do Solo v3 (regeneracao gerador corrigido)"
15
16
17 def main():
18     port = int(os.getenv("WEBSOCKET_PORT", "5025"))
19     host = os.getenv("WEBSOCKET_HOST", "localhost")
20     print(f"🚀 {PROJECT_NAME} - WebSocket server em ws://{host}:{port}")
21     asyncio.run(run_websocket_server(host=host, port=port))
22
23
24 if __name__ == "__main__":
25     main()
26
```

Salvar Arquivo

Baixar ZIP

Executar

Etapa 6 — Código gerado (81 arquivos)